ONE-PAGE REPO SUMMARY

Baylio

AI call assistant for auto repair shops

Generated from local repo evidence only
Single-page executive + technical brief

WHAT IT IS

Baylio is a multi-tenant SaaS platform that installs an AI receptionist for auto repair shops. The codebase combines a React/Vite portal suite with an Express/tRPC API that handles Twilio calls, ElevenLabs voice agents, call analytics, integrations, and Stripe billing.

WHO IT'S FOR

Primary: auto repair shop owners/operators. **Secondary:** partner users and admins, based on partner/admin routes.

CURRENT SIGNALS

Status: README says MVP v2.1. **Pricing:** README lists $199/$349/$599 tiers plus setup fee.

WHAT IT DOES

- Answers inbound calls via Twilio and connects callers to ElevenLabs conversational AI.
- Captures call context: caller, vehicle, requested service, appointment intent, and upsell opportunities.
- Gives shop owners dashboards for shops, call logs, scorecards, analytics, missed-call audits, costs, and notifications.
- Configures agent behavior: voice/personality, business hours, service catalog, shop settings, and Twilio provisioning.
- Runs post-call workflows for transcription, classification, usage metering, recaps, and reporting.
- Handles commercial plumbing: Stripe checkout, billing portal, subscriptions, usage records, setup fees, and webhooks.
- Supports integrations surfaced in code for Google auth/calendar/sheets, HubSpot, Shopmonkey, SMS, and email.

HOW IT WORKS

React 19 + Vite SPA: main app, partners portal, admin portal

v

tRPC client calls Express `/api/trpc` domain routers

v

Routers delegate to services: Twilio, ElevenLabs, prompt compiler, post-call pipeline, Stripe, integrations

v

Drizzle data layer with PostgreSQL driver/config and 17 pgTable declarations

- **Live call path:** Twilio `/api/twilio` -> rate limit/signature validation -> shop context cache -> ElevenLabs WebSocket -> async post-call updates.
- **Billing path:** Stripe `/api/stripe` webhook router updates subscription and usage state.
- **Tenant model:** repo docs/code use owner/user-scoped access rather than database-native RLS.

REPO EVIDENCE HIGHLIGHTS

Frontend
Route-level pages for landing, onboarding, dashboard, shops, calls, analytics, subscriptions, integrations, help, partner, and admin flows.

Backend
Express app wires tRPC, Twilio, Stripe, Google OAuth, rate limiting, webhook validation, and static/Vite serving.

Data model
Users, organizations, shops, agent configs, call logs, audits, subscriptions, usage, notifications, partners, referrals, callers, contact submissions, integrations.

Known mismatch
Current Drizzle config/server DB driver are PostgreSQL; older docs still mention MySQL/TiDB and earlier table counts.

HOW TO RUN

1. Install repo prerequisites from docs: Node.js 20+ and pnpm 10+.
2. Run `pnpm install` from the repo root.
3. Create `.env`. Docs list `DATABASE_URL`, `JWT_SECRET`, Twilio, ElevenLabs, and Stripe keys; code also references `SUPABASE_*` and `ANTHROPIC_API_KEY`.
4. Run `pnpm dev`. Other package scripts: `pnpm build`, `pnpm test`, `pnpm db:push`.

! **Not found in repo:** canonical `.env.example`. Docs say copy secrets from the team vault. Port docs conflict: code defaults to `3000`; setup docs say `5000`.

Evidence checked: README.md, package.json, docs/DEV_SETUP.md, docs/ARCHITECTURE.md, client/src/App.tsx, server/_core/index.ts, server/routers.ts, server/db.ts, drizzle.config.ts, drizzle/schema.ts, and source tree names. No external sources used.